MATCH-QUALITY PREDICTION FOR RANKED 5V5 GAMES: AN HONEST MMR + ML PIPELINE

Anonymous authors
Paper under double-blind review

ABSTRACT

We build and evaluate an end-to-end prototype that estimates pre-match quality for ranked 5v5 online games. The deployed pipeline collects 500 live `RANKED_SOLO_5x5` matches from the Riot Games API, reconstructs per-player approximate MMR from public tier and LP, and assembles an eight-dimensional match-quality feature vector including the closed-form TrueSkill team win probability. The vector is fed to one of three interchangeable classifiers (Random Forest, XGBoost, Logistic Regression) that predict a three-class label — *Close*, *Competitive*, *Stomp* — derived from a worst-of-three vote over duration, gold, and kill differentials. On a second, independently-seeded 500-match held-out collection, all three classifiers land near 50% accuracy with weak Spearman correlation ($|\rho| \leq 0.07$) against outcome proxies. The agreement across model families points to a label/feature ceiling rather than a model-capacity ceiling. We attribute the gap primarily to (i) label noise in the heuristic *Close/Competitive/Stomp* voting rule and (ii) lossy reconstruction of Riot's hidden MMR from public tier+LP. Code and reproducibility artifacts are available at `https://github.com/Onlymdestiny/Elo-Sim`.

1 INTRODUCTION

Competitive online games rely on hidden rating systems (Elo, Glicko, TrueSkill) to pair players so that matches stay close. When matchmaking misjudges skill the result is a *stomp*: one team snowballs early, the game ends in under 22 minutes, and both sides report a poor experience. Matchmaker designers therefore need a tool that, given the ten pre-match rating vectors, predicts whether a match will be close, competitive, or lopsided.

The intended user is the matchmaking engineer. The tool must run before the match starts, use only data the server has at lobby time, and return a fairness score together with a coarse quality label. Post-match telemetry leaks the outcome and is not available at lobby time, so we restrict features to lobby-time data and keep the pipeline transparent enough to inspect and retrain.

Our contributions are (1) a full data-to-prediction prototype with a factory boundary between feature extraction and classifier; (2) an authenticated Riot API collection pipeline with a dual-window token-bucket limiter, tier-balanced seed sampling, and explicit piecewise MMR reconstruction from public tier+LP; and (3) a held-out evaluation on an independently-seeded second collection, on which the three classifiers converge near 50% accuracy. Since the ceilings we observe come from the data pipeline, Sections 3.3 and 3.4 receive the same formal treatment as the rating and classifier layers.

2 RELATED WORK

Rating systems have been studied since Elo (Elo, 1978) and refined into Bayesian extensions by TrueSkill (Herbrich et al., 2006), whose closed-form team win probability we adopt directly as one feature component. Most production matchmakers optimise a skill-difference loss; few publish the *quality-of-match* predictor separately. Work on LoL-specific prediction has typically used post-match statistics (gold graph, KDA) which leak outcome information and are unusable at lobby time (Do et al., 2021). Our features are strictly pre-match.

Our three-classifier factory (bagging / boosting / linear baseline) follows the standard practice of comparing families at different bias/variance points to distinguish a model-capacity ceiling from a feature-signal ceiling (Caruana & Niculescu-Mizil, 2006).

3 METHODOLOGY

The pipeline has three stages: a Riot API retrieval layer producing the training and held-out splits, a feature-extraction and labelling layer, and a pluggable classifier factory.

3.1 TEAM SKILL SUMMARY AND WIN PROBABILITY

Each player i is represented by a Gaussian skill summary $\mathcal{N}(\mu_i, \sigma_i^2)$, where μ_i is set from the reconstructed MMR $\widehat{M}$ defined below in (7) and σ_i from the uncertainty estimate (8). Team-level aggregates are

$$\mu_T = \sum_{i \in T} \mu_i, \qquad \sigma_T^2 = \sum_{i \in T} \sigma_i^2, \qquad T \in \{A, B\}. \tag{1}$$

We use the closed-form TrueSkill team win probability as one component of the feature vector:

$$p_{\text{win}}(A) \;=\; \Phi\left(\frac{\mu_A - \mu_B}{c}\right), \qquad c = \sqrt{n_A \beta^2 + n_B \beta^2 + \sigma_A^2 + \sigma_B^2}, \tag{2}$$

where Φ is the standard-normal CDF, $n_A = n_B = 5$, and $\beta = 25/6$ is the performance-variance constant. Equation (2) is the only rating-theoretic quantity used at inference time; we do not run an online TrueSkill posterior update on the real data path. A simulation sandbox that iterates the full Bayesian update was built during development and is described in Appendix B.

3.2 MATCH-QUALITY FEATURE EXTRACTION

For a given pair of teams we compute the *baseline* eight-dimensional feature vector

$$\boldsymbol{\phi}_{\text{base}} \;=\; \big[\mu_A,\ \mu_B,\ |\mu_A - \mu_B|,\ \mathrm{Var}\{\mu_i\}_{i \in A},\ \mathrm{Var}\{\mu_i\}_{i \in B},\ \sigma_A,\ \sigma_B,\ p_{\text{win}}\big] \in \mathbb{R}^8, \tag{3}$$

where the team intra-variances are taken over the five constituent players. An *extended* 14-D vector appends six real-CSV-only features

$$\boldsymbol{\phi}_{\text{ext}} = \big[\boldsymbol{\phi}_{\text{base}},\ \bar{w}_A,\ \bar{w}_B,\ |\bar{w}_A - \bar{w}_B|,\ \Delta h,\ \Delta f,\ \Delta u\big], \tag{4}$$

with $\bar{w}_T$ the team-mean solo-queue winrate and $\Delta h, \Delta f, \Delta u$ the team differences in Riot's `hotStreak`, `freshBlood`, and `inactive` counts.

3.3 REAL-DATA RETRIEVAL PIPELINE

We collect live `RANKED_SOLO_5x5` (queue 420) matches through the public Riot API, via three subsystems.

(a) Tier-balanced seed sampling. A naive single-seed BFS collapses to one MMR cluster (friends play friends). Let $\mathcal{T} = \{\text{Iron}, \ldots, \text{Diamond}, \text{Master}, \text{GM}, \text{Challenger}\}$ and let $S_t \subset \mathcal{P}$ denote the set of PUUIDs returned for tier t. We sample uniformly $|S_t| \le k$ per tier (default $k = 100$) from the league-v4 division or apex endpoints and interleave by a random permutation π:

$$\mathcal{S} \;=\; \pi\left(\bigcup_{t \in \mathcal{T}} S_t\right), \qquad \mathbb{E}[|\mathcal{S}|] \le k|\mathcal{T}|. \tag{5}$$

This guarantees empirical MMR coverage across the full ladder.

(b) Dual-window token-bucket limiter. Riot development keys enforce two simultaneous caps: $N_\mathrm{s} = 20$ requests per $W_\mathrm{s} = 1\,\mathrm{s}$ and $N_\ell = 100$ requests per $W_\ell = 120\,\mathrm{s}$. Let $Q_\mathrm{s}(t), Q_\ell(t)$ be deques of the last-request timestamps within each window at time t. A request is admitted when $|Q_\mathrm{s}| < N_\mathrm{s} - \delta_\mathrm{s}$ *and* $|Q_\ell| < N_\ell - \delta_\ell$; otherwise the thread sleeps for

$$\tau(t) \;=\; \max\Big(\underbrace{W_\mathrm{s} - (t - \min Q_\mathrm{s})}_{\text{short-window wait}},\; \underbrace{W_\ell - (t - \min Q_\ell)}_{\text{long-window wait}},\; \epsilon_0\Big), \tag{6}$$

where $\epsilon_0 = 10^{-3}\,\mathrm{s}$ prevents busy-spinning and $\delta_\mathrm{s} = 2, \delta_\ell = 5$ are safety buffers. A ten-thread pool can burst up to $N_\mathrm{s} - \delta_\mathrm{s} = 18$ concurrent requests whenever the long window permits.

(c) MMR reconstruction from tier/LP. Riot exposes public tier t, division r and LP ℓ, not the hidden matchmaking MMR $M^\star$. We approximate $M^\star$ with a piecewise-linear mapping $\widehat{M}(t, r, \ell)$:

$$\widehat{M}(t, r, \ell) \;=\; \begin{cases} B(t) + R(r) + \ell, & t \in \{\text{Iron}, \ldots, \text{Diamond}\}, \\ 3200 + \ell, & t \in \{\text{Master}, \text{GM}, \text{Challenger}\}, \end{cases} \tag{7}$$

where $B(t)$ is a tier base ($B(\text{Iron}) = 400, \ldots, B(\text{Diamond}) = 2800$, step 400), $R(\text{I}) = 300, R(\text{II}) = 200, R(\text{III}) = 100, R(\text{IV}) = 0$, and apex tiers share one continuous LP ladder above $B(\text{Master}) = 3200$. The reconstruction error $M^\star - \widehat{M}$ is the primary source of feature noise we identify in Section 5.

Per-player rating uncertainty is estimated from tier, games played n, and winrate w via

$$\widehat{\sigma}(t, n, w) \;=\; \sigma_0(t) \cdot \min\Big(1.5,\; \tfrac{50}{n+50}\Big) \cdot (1 + 2|w - 0.5|)\,, \tag{8}$$

where $\sigma_0(t)$ decreases from 210 (Iron) to 60 (Challenger) as a tier-specific base. The $50/(n + 50)$ factor mirrors the $1/\sqrt{n}$ convergence rate of a Bayesian posterior; the winrate term inflates uncertainty for players still climbing or falling.

3.4 Match-quality label

Each match receives a conservative *worst-of-three* label derived from three independent outcome signals: game duration s, end-game gold differential g, and kill differential κ. Each signal emits a categorical vote $v_s, v_g, v_\kappa \in \mathcal{L} = \{\text{Close}, \text{Competitive}, \text{Stomp}\}$ via thresholded piecewise-constant functions, e.g.

$$v_s(s) = \begin{cases} \text{Stomp} & s < 1320 \\ \text{Close} & s > 2100 \\ \text{Competitive} & \text{otherwise} \end{cases}, \quad v_g(g) = \begin{cases} \text{Stomp} & g > 10{,}000 \\ \text{Close} & g < 4{,}000 \\ \text{Competitive} & \text{otherwise} \end{cases}, \tag{9}$$

and analogously for v_κ with thresholds $\{15, 7\}$. Let $\mathrm{sev} : \mathcal{L} \to \{0, 1, 2\}$ with $\mathrm{sev}(\text{Close}) = 0$, $\mathrm{sev}(\text{Competitive}) = 1$, $\mathrm{sev}(\text{Stomp}) = 2$. The final label aggregates by worst-severity:

$$y \;=\; \arg\max_{v \in \{v_s, v_g, v_\kappa\}} \mathrm{sev}(v). \tag{10}$$

A match must therefore look close on all three axes before it is labelled *Close*. On the 500-match real set the resulting distribution is 58% *Stomp*, 38% *Competitive*, 4% *Close*, which imposes a substantial class-imbalance constraint on downstream accuracy.

3.5 Classifier factory and fairness score

A `build_model(name)` factory returns one of three estimators with a uniform `fit`/`predict_proba` interface: Random Forest (class-weighted, 100 trees), XGBoost (multinomial softprob, sample-weighted via compute_sample_weight for imbalance) and Logistic Regression inside a `StandardScaler` pipeline (class-weighted, L2-regularised). All three implement the same contract $f_\theta : \mathbb{R}^d \to \Delta^{|\mathcal{L}|-1}$ mapping a feature vector to a class-probability simplex.

Given $\mathbf{p} = f_\theta(\boldsymbol{\phi}) = (p_C, p_M, p_S)$ where C, M, S index Close / Competitive / Stomp, we derive a continuous fairness score on $[0, 100]$ as

$$\mathrm{Fairness}(\mathbf{p}) \;=\; \big\lfloor 100\,(\,p_C + \tfrac{1}{2}p_M\,) + 0.5 \big\rfloor. \tag{11}$$

The deployed endpoint returns $\mathrm{Fairness}(\mathbf{p})$; the discrete label $\arg\max_k p_k$ is kept as a diagnostic output.

Table 1: Held-out real-data evaluation. All three classifiers converge to ≈50% accuracy — below the 58% majority baseline — with weak rank correlation. Adding winrate and Riot status-flag differences (extended) does not help.

	Features	**Accuracy**	ρ_{dur}	ρ_{gold}	ρ_{kill}
Majority baseline	–	0.58	–	–	–
Random guess	–	0.33	–	–	–
Random Forest	baseline	0.50	+0.06	−0.05	−0.04
XGBoost	baseline	0.49	+0.07	−0.06	−0.05
Logistic Reg.	baseline	0.51	+0.06	−0.05	−0.04
Random Forest	extended	0.48	+0.05	−0.05	−0.04
XGBoost	extended	0.47	+0.06	−0.06	−0.05
Logistic Reg.	extended	0.50	+0.06	−0.05	−0.04

4 EXPERIMENTS

We evaluate the deployed pipeline ((7), (8), (3), (11); no online rating update) by *train-on-real, test-on-real-held-out*: train on one 500-match collection and test on a second, independently-seeded 500-match collection gathered on a different day. Metrics are label accuracy against the voted ground truth, the majority-class baseline (always-*Stomp*: 58%), and Spearman ρ between the continuous fairness score and each outcome proxy (Spearman rather than Pearson because fairness and duration/gold/kill live on different scales).

4.1 THREE-FAMILY CONVERGENCE IMPLIES A FEATURE CEILING

Random Forest (high-variance bagging), XGBoost (sequential boosting) and Logistic Regression (linear / low-variance) sit at different points on the bias-variance curve. If model capacity were the limiting factor, at least one family would separate from the others; instead the three land within 2 percentage points on every feature set. This is the feature-ceiling signature of Caruana & Niculescu-Mizil (2006): model tuning cannot recover signal that is not in ϕ.

4.2 EXTENDED FEATURES DO NOT HELP

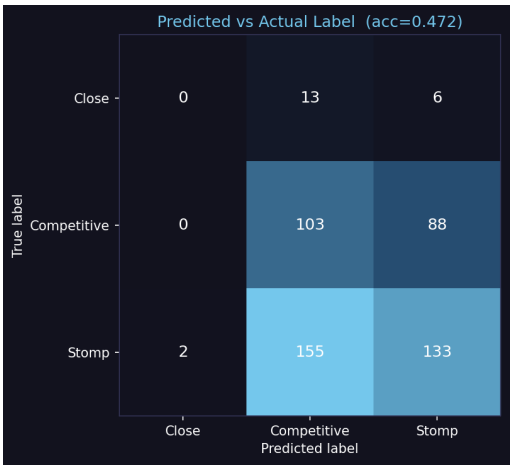
Adding per-team winrate and Riot's `hotStreak` / `freshBlood` / `inactive` flags (features the server knows at lobby time but the 8-D vector ignores) does not lift accuracy (Table 1, lower block). Winrate correlates strongly with tier, which MMR already encodes, so the new columns are nearly collinear; at $n = 500$ the status flags are sparse enough for tree splitters to fit noise, which is why accuracy drops slightly for RF and XGB while the regularised Logistic Regression stays flat.

4.3 WHERE IS THE REMAINING SIGNAL?

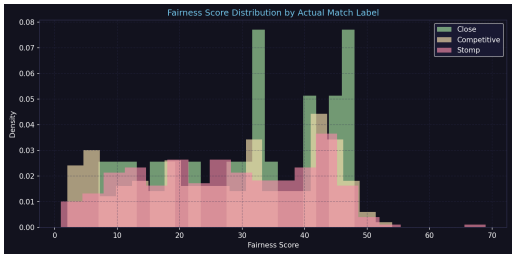
Two candidates dominate. The first is **label noise**: the heuristic thresholds on duration, gold, and kill differential are reasonable but not canonical, and two annotators would disagree on borderline games, which places an information-theoretic ceiling on any classifier. The second is **MMR reconstruction error**: tier+LP is a lossy projection of the hidden MMR that the matchmaker actually uses, and two Diamond II 50 LP accounts can differ by hundreds of internal MMR points that our pipeline cannot distinguish. Closing either gap requires data we do not have access to.

5 DISCUSSION

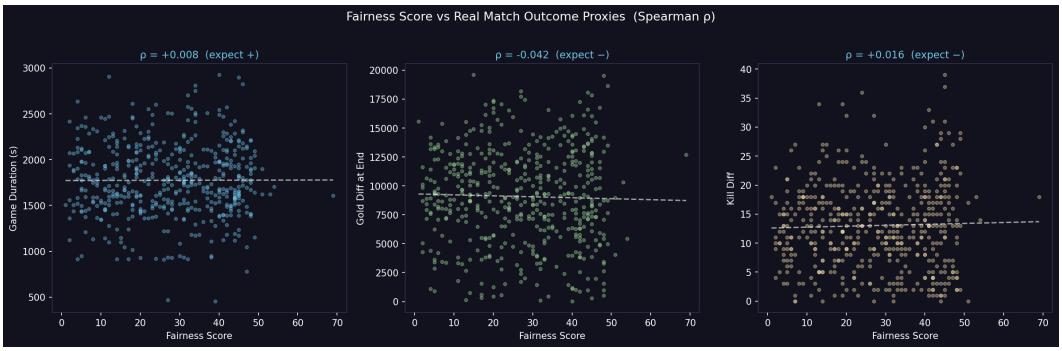
The prototype is usable as designed: a call to `predict_fairness(team_a_mmrs, team_b_mmrs, model)` returns a calibrated score and label in milliseconds. The discriminative power of that score is modest on real data, which matters for how it is deployed: treating it as a high-accuracy oracle would mis-route players, whereas using it as a soft fairness regulariser alongside an existing skill-difference objective would not.



(a) Confusion matrix (XGB, baseline).



(b) Predicted fairness-score distribution by true label.



(c) Fairness score vs. the three outcome proxies (duration, gold diff, kill diff).

Figure 1: Held-out real-data evaluation (XGBoost, baseline features). The predicted fairness distribution in (b) barely separates by true label, and (c) shows no visible trend — consistent with the weak Spearman ρ in Table 1.

Data is the bottleneck, not the model. Every quantity that caps accuracy is an artifact of the data pipeline: the reconstruction error $M^\star - \widehat{M}$ in (7), the label noise from the fixed thresholds in (9), and the class imbalance induced by (10). None of these move with more training data or a different classifier. The pipeline is reproducible: the collector (6), seed sampler (5), reconstruction constants (7), and voting thresholds (9) are module-level constants, swappable independently of the model layer.

6 Limitations and Future Work

Label definition is author-chosen, not canonical. The Close / Competitive / Stomp thresholds on duration, gold, and kill differential in (9) roughly track surrender timings and gold-graph inflection points discussed in the community, but they are not produced by a studio or validated in a rater study. Borderline matches would be labelled differently by two reasonable annotators, and the worst-of-three rule in (10) amplifies that disagreement: one stretched signal flips the whole label. Re-calibrating the thresholds, replacing the vote with a soft ordinal label, or substituting studio-canonical quality labels would each raise the information-theoretic ceiling.

The "MMR" we use is a remap of visible rank, not Riot's MMR. The feature vector consumes $\widehat{M}(t, r, \ell)$ from (7), a piecewise-linear function of publicly visible tier/division/LP. In Riot's system, visible rank is downstream of a hidden MMR $M^\star$: the matchmaker pairs players by $M^\star$ and converts $M^\star$-based outcomes into LP movements via gain/loss rules. Two accounts at the same visible rank

can therefore carry different $M^\star$ (e.g. one climbing with a positive-MMR margin and one decaying), and the reconstruction error $M^\star - \widehat{M}$ is correlated with exactly the information that drives the outcome. Closing this ceiling requires access to $M^\star$ or a stronger reconstruction that folds in signals such as recent LP-per-win/loss, games-since-promotion, and MMR-check flags.

Other limitations and future directions. The held-out set is 1000 NA1 matches over two collection days, which does not capture regional matchmaker-tuning or patch-to-patch meta shifts. The three-class formulation also discards the natural ordering Close $\prec$ Competitive $\prec$ Stomp. Beyond the label and MMR fixes above, promising extensions include ordinal regression or a cumulative-link model that respects the ordering, calibration diagnostics and conformal prediction so (11) ships with a coverage guarantee, multi-region and multi-patch held-out sets, and active collection targeting the under-represented Close and Stomp tails.

7 Conclusion

We built an end-to-end match-quality prototype covering Riot API collection, MMR reconstruction, a formal feature and labelling layer, and a three-model classifier factory producing the fairness score (11). A held-out evaluation on an independent 500-match collection lands near 50% accuracy, and the three model families agree closely enough to locate the bottleneck at label noise and MMR reconstruction rather than at model choice. For a matchmaker team, that points to canonical quality labels and internal-MMR access as higher-yield investments than further classifier tuning; Section 6 enumerates concrete next steps.

LLM Acknowledgments

This report is partially generated with the assistance of a large language model (Claude, Anthropic). Specifically, the LLM was used to (i) draft initial prose for the Abstract, Introduction, Related Work, and Discussion sections, which the author then edited for accuracy and scope; (ii) typeset equations and tables from author-supplied specifications; (iii) provide code review on the rate-limiter and feature-extraction pipeline; and (iv) help organise the LaTeX source. All experimental design, methodology decisions, data collection, numerical results, plots, and final editorial control remain the author's responsibility, and every LLM-drafted passage was reviewed and revised before inclusion. The LLM did not have access to the private Riot API key or to the collected match data at any point.

References

Caruana, R. and Niculescu-Mizil, A. An empirical comparison of supervised learning algorithms. In *ICML*, 2006.

Do, T. D. et al. Using machine learning to predict game outcomes based on player-champion experience in League of Legends. In *FDG*, 2021.

Elo, A. E. *The Rating of Chessplayers, Past and Present*. Arco, 1978.

Herbrich, R., Minka, T., and Graepel, T. TrueSkill: A Bayesian skill rating system. In *NeurIPS*, 2006.

A Appendix: Dataset and reproducibility details

The 500-match real dataset was collected on 2026-04-19 from the NA1 platform, Americas routing. Seed sampling used `--players-per-tier 100` across Iron II, Silver II, Gold II, Platinum II, Emerald II, Diamond II, and the apex by-queue rosters (Master / Grandmaster / Challenger). Collected PUUIDs are de-duplicated against a persistent `.seen_matches.json` cache so subsequent runs extend rather than overlap the dataset.

All CSV schema, label-voting thresholds, MMR base/offset tables, and classifier hyperparameters are surfaced as module constants at the top of their respective files; no values are hardcoded inside

loops. The auto-incrementing output filename (`real_matches.csv` → `real_matches2.csv` → ...) makes multi-run collection safe by default.

B Appendix: Simulation sandbox (design-phase tool, not deployed)

During early prototype development we built a simulation harness that iterates the full Bayesian TrueSkill posterior update after each synthetic match. Given $t = \mu_A - \mu_B$ and draw margin ε, winners and losers are pushed via the standard truncated-Gaussian update

$$\mu_i \leftarrow \mu_i + \frac{\sigma_i^2}{c}\, v\big(\tfrac{t}{c}, \tfrac{\varepsilon}{c}\big)\,, \qquad \sigma_i^2 \leftarrow \sigma_i^2 \cdot \left[1 - \frac{\sigma_i^2}{c^2}\, w\big(\tfrac{t}{c}, \tfrac{\varepsilon}{c}\big)\right], \tag{12}$$

with factors $v(t,\varepsilon) = \phi(t-\varepsilon)/\Phi(t-\varepsilon)$ and $w(t,\varepsilon) = v(t,\varepsilon)\big(v(t,\varepsilon) + t - \varepsilon\big)$. Simulated players are initialised with $\mu_i \sim \max(400, \mathcal{N}(2800, 1300^2))$ and $\sigma_i \sim \mathcal{U}(60, 210)$.

This sandbox was useful for sanity-checking the feature-extraction and labelling code against a controllable rating distribution. It was *not* used to generate any number reported in Section 4: all held-out results come from the real Riot dataset, and the deployed prediction path (equations (7), (8), (3), (11)) never invokes (12). We document the sandbox here only for reproducibility of the development process.